-
-
- **表单——表单提交**
  - HTML 标签本身具有数据提交的能力，然而现实中更常见的是通过监听 DOM 事件获取表单的数据，然后通过 Ajax 将表单的数据提交至服务端。

```
<form>
  昵称: <input type="text" name="nickname">
  <br>
  密码: <input type="password" name="password">
  <br>
  <button>提交</button>
</form>
<script src="your/path/jquery-版本号.min.js"></script>
<script>
 // 监听表单的 submit 事件
  $('form').submit(function (ev) {

    // 表单提交的相关逻辑...

    // 阻止 HTML 表单的默认提交
    // ev.preventDefault();

    // ev.preventDefault() 在 jQuery 中可以
    // 被简写成 return false
    return false;
  })
</script>
```

  - 总结：
    - HTML 本身具有提交表单的能力，但是会刷新当前的页面
    - 通过监听 `form` 元素的 `submit` 事件提交表单能够带来较好的用户体验
    - 用户按回车键或单击提交按钮（`type` 属性值为 `submit`）时会触发 `form` 的 `submit` 事件
    - jQuery 的事件回调函数中执行 `return false` 时，相当于执行 `ev.preventDefault()`
    - 注：`button` 标签默认的 `type` 属性为 `submit`

- **表单——序列化**
  - HTML 标签默认提交表单时会自动根据表单项的 `name` 属性将用户在表单中填写的数据提交至服务端，然而采用监听 DOM 事件对表单提交时，需要开发者自行获取表单项中用户填写的数据，其用户如下代码所示：

```
<form>
  昵称: <input type="text" name="nickname">
  <br>
  密码: <input type="password" name="password">
  <br>
    <button>提交</button>
</form>
<script src="your/path/jquery-版本号.min.js"></script>
<script>
  // 监听表单的 submit 事件
  $('form').on('submit', function () {

    // 通过 jQuery 对象的 serialize 方法，获取所有表单元素的数据
    $(this).serialize();

        // 阻止表单默认提交
    return false;
  })
</script>
```

- 总结：
  - `val` 方法用来获取或设置表单元素的值
  - `serialize` 方法用来获取表单中具有 `name` 属性的表单项中的数据
  - 调用 `serialize` 方法获取到的表单数据会被处理 `key=val&key1=val1&key2=val2&...` 格式的字符串
  - `trim` 方法用来清除字符串两端的空白符
  - 注：单选框和复选框表单项的 `value` 属性省略时默认值为 `on`

- **插件——插件机制**
  - 插件是 jQuery 的扩展机制，它本质上是 jQuery 原型对象的一个方法，如下代码所示：

```
<p>调用插件方法 color 后，改变文本的颜色。</p>
<script src="your/path/jquery-版本号.min.js"></script>
<script>
  // 1. 在原型对象中添加的 color 方法，且所有 jQuery 实例均可以调用
  $.fn.color = function (val) {
    console.log('我的第一个插件...');

    // 接收传入的参数，设置文件颜色
```

```
    $(this).css('color', val);
  }

  // 可以查看到原型中被添加了 color 方法
  console.log($.fn);

  // 2. 通过 jQuery 实例调用插件方法
  $('p').color('red');
</script>
```

- 总结：
  - 通过 `$.fn` 为原型对象添加插件方法
  - 插件方法中的 `this` 指向调用此插件的 DOM 元素
  - 一般情况下插件方法都有默认参数，传入自定义参数会覆盖内置默认参数
  - `extend` 方法用于合并多个对象，遇到相同的属性名或方法名时则会出现替换情况
  - 注：根据需要有时插件方法也会被添加到 jQuery 构造函数上成为一个静态方法

- **插件——表单验证**
  - `jquery-validate` 是一个基于 jQuery 的表单验证插件，其基本用法需要从 4 个方面入手：
  - 是通过自定义属性约束表单项的验证规则，如下所示：

自定义属性	含义	备注
data-required	验证表单项不能为空	不需要有值
data-pattern	基于正则表达式验证	

```
<form>
  昵称: <input type="text" data-required name="nickname">
  <br>
  密码: <input type="password" data-pattern="[a-z0-9]{6,}" name="pass">
  <br>
  <button>提交</button>
</form>
<script src="yourpath/jquery/jquery-3.5.1.min.js"></script>
<!-- jQuery 插件必须在 jQuery 之后被引入 -->
<script src="yourpath/jquery-validate/jquery-validate.js"></script>
<script>
  // 基本用法
```

```
    $('form').validate({
    // 一些参数...
  });
</script>
```

- 通过插件方法的参数设定触发验证的时机，如下所示：

验证时机	含义	备注
onBlur	失去焦点时验证	
onSubmit	提交表单时验证	默认
onChange	表单内容改变时验证	
onKeyup	按钮时验证	
sendForm	是否禁用表单默认提交	

```
<script src="yourpath/jquery/jquery-3.5.1.min.js"></script>
<!-- jQuery 插件必须在 jQuery 之后被引入 -->
<script src="yourpath/jquery-validate/jquery-validate.js"></script>
<script>
  // 基本用法
    $('form').validate({
    onBlur: true,
    // 提交表单时不再进行验证
    onSubmit: false,
    onKeyup: true
  });
</script>
```

- 为用户提示错误信息，如下所示：

自定义属性	含义	备注
data-describedby	指定显示错误信息的标签	值为标签的 ID
data-description	指定错误信息的内容（标签）	

```html
<form>
  昵称:
  <input
    type="text"
    data-describedby="nickname-error"
    data-description="nickname"
    data-required
    name="nickname">
  <span id="nickname-error"></span>
  <br>

  密码:
  <input
    type="password"
    data-describedby="pass-error"
    data-description="pass"
    data-pattern="[a-z0-9]{6,}"
    name="pass">
  <span id="pass-error"></span>
  <br>
  <button>提交</button>
</form>
<script src="yourpath/jquery/jquery-3.5.1.min.js"></script>
<!-- jQuery 插件必须在 jQuery 之后被引入 -->
<script src="yourpath/jquery-validate/jquery-validate.js"></script>
<script>
  // 基本用法
    $('form').validate({
    description: {
      nickname: {
        required: '昵称不能为空!'
      },
      pass: {
        required: '密码不能为空!',
        pattern: '密码只能为数字和字母且不能少于6位!'
      }
    }
  });
</script>
```

- 所有表单项的数据验证通过后提交表单，通过 send如下所示：

回调函数	含义	备注
valid	所有表单项验证通过时被执行	
invalid	至少一个表单项未通过验证时执行	

```
<form>
  昵称:
  <input type="text" data-required name="nickname">
  <br>
  密码:
  <input type="password" data-pattern="[a-z0-9]{6,}" name="pass">
  <br>
  <button>提交</button>
</form>
<script src="yourpath/jquery/jquery-3.5.1.min.js"></script>
<!-- jQuery 插件必须在 jQuery 之后被引入 -->
<script src="yourpath/jquery-validate/jquery-validate.js"></script>
<script>
  // 基本用法
    $('form').validate({
        sendForm: false,
    valid: function () {
      // 表单通过验证后执行该回调函数

      // 获取用户在表单中填写的数据
      $(this).serilize();
    },
    invalid: function () {
      // 表单未通过验证时执行该回调函数
    }
  });
</script>
```

- 更多的配置参数详见官方文档。

- **插件——轮播图**

- 基于 jQuery 的轮播图插件非常多，本次课程中以 [slick](http://kenwheeler.github.io/slick/) 插件作为学习目标，其用法如下所示：
- 首先 slick 插件要求 HTML 必须满足以下结构，如下代码所示：

```
<div class="your-class">
  <div>轮播元素一</div>
  <div>轮播元素二</div>
  <div>轮播元素三</div>
</div>
```

- 其次引入 slick 插件及与插件配套的样式表，如下代码所示：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 轮播图插件</title>
  <!-- 引入插件所需要的样式表 -->
  <link rel="stylesheet" href="./assets/slick/slick.css">
  <link rel="stylesheet" href="./assets/slick/slick-theme.css">
</head>
<body>
  <!-- 父子关系的 HTML 结构 -->
  <div class="slider">
    <div>轮播元素一</div>
    <div>轮播元素二</div>
    <div>轮播元素三</div>
  </div>
  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入插件 -->
  <script src="./assets/slick/slick.min.js"></script>
  <script>
    $('.slider').slick({
      autoplay: true,
      arrows: false
    });
  </script>
</body>
```

```
</html>
```

- 最后调用 slick 插件方法，如下代码所示：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 轮播图插件</title>
  <!-- 引入插件所需要的样式表 -->
  <link rel="stylesheet" href="./assets/slick/slick.css">
  <link rel="stylesheet" href="./assets/slick/slick-theme.css">
</head>
<body>
  <!-- 父子关系的 HTML 结构 -->
  <div class="slider">
    <div>轮播元素一</div>
    <div>轮播元素二</div>
    <div>轮播元素三</div>
  </div>
  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入插件 -->
  <script src="./assets/slick/slick.min.js"></script>
  <script>
    // 调用插件方法
    $('.slider').slick({
      autoplay: true,
      arrows: false
    });
  </script>
</body>
</html>
```

- slick 提供了许多参数用于自定义轮播图的执行效果，以下是对常用参数的使用说明：

配置参数	含义	备注
autoplay	轮播效果自动执行	
arrows	是否显示翻页按钮	
prevArrow	自定义【上一页】按钮	
nextArrow	自定义【下一页】按钮	
dots	是否显示指示器	

- 更多的配置参数详见官方文档，另外当插件提供的样式与设计稿不符时，可通过自定义 CSS 覆盖默认样式实来实现。

- **插件——图片裁切**
  - 图片裁切是网页中比较常见的功能，完全自主实现该功能是十分复杂的，不过有一些第三方封装好的 jQuery 插件可供我们使用，本次课以 [Jcrop](http://deepliquid.com/content/Jcrop.html) 为学习目标，其用法如下：
  - 首先引入 Jcrop 插件及配套的样式：

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 图片裁切插件</title>
  <!-- 引入 Jcrop 配套的样式 -->
  <link rel="stylesheet" href="./assets/jcrop/css/jquery.Jcrop.min.css">
</head>
<body>

  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入 Jcrop -->
  <script src="./assets/jcrop/js/jquery.Jcrop.min.js"></script>
</body>
</html>
```

  - 其实调用 Jcrop 插件方法：

```html
<!DOCTYPE html>
```

```html
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 图片裁切插件</title>
  <!-- 引入 Jcrop 配套的样式 -->
  <link rel="stylesheet" href="./assets/jcrop/css/jquery.Jcrop.min.css">
</head>
<body>
  <!-- 待裁切的图片 -->
  <img src="./images/xiaohei.jpg" alt="">

  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入 Jcrop -->
  <script src="./assets/jcrop/js/jquery.Jcrop.min.js"></script>
  <script>
    // 调用插件方法即可!
    $('img').Jcrop();
  </script>
</body>
</html>
```

- Jcrop 提供了许多参数用于自定义图片裁切的细节，以下是对常用参数的使用说明：

配置参数	含义	备注
aspectRatio	限制裁切结果区域的宽高比	
setSelect	初始裁切区域的坐标	左上角及右下角坐标
boxWidth	指定图片的裁切时的宽度	确保获得真实裁切尺寸及坐标
onSelect	调整裁切区域后执行的回调函数	用于获取裁切区域的坐标及尺寸

- 更多的配置参数详见官方文档，另外必须要知道的是 Jcrop 并未真正对图片进行裁切，它仅仅是**计算并标识**出了待裁切区域的位置及尺寸，通常计算得到的位置及尺寸发送至后端由后端根据这些坐标及尺寸完成裁切。
- 注：Jcrop 自从 v0.9.12 以后（不包括 v0.9.12）独立成一个专门的图片裁切的库，可以与 jQuery、Vue、React 等配合使用。

- **插件——日期选择器**
  - 在网页中为用户提供方便快捷的日期选择方式，不仅有利于增强用户体验，最重要的是能够保证日期的准确性及统一性，[datepicker](https://fengyuanchen.github.io/datepicker/) 提供了良好的选择日期的方式，其用法如下所示：

- 首先引入 datepicker 插件及配套样式：

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 日期选择器</title>
  <!-- 插件配套的样式 -->
  <link rel="stylesheet" href="./assets/datepicker/datepicker.min.css">
</head>
<body>
  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入插件 -->
  <script src="./assets/datepicker/datepicker.min.js"></script>
</body>
</html>
```

- 其次调用 datepicker 插件方法，输入框获得焦点时自动弹出日期选择层：

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 日期选择器</title>
  <!-- 插件配套的样式 -->
  <link rel="stylesheet" href="./assets/datepicker/datepicker.min.css">
</head>
<body>
  <h3>选择日期</h3>

  <input type="text" class="datepicker">

  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入插件 -->
  <script src="./assets/datepicker/datepicker.min.js"></script>
  <script>
    // 调用插件方法
```

```
    $('.datepicker').datepicker();
  </script>
</body>
</html>
```

- 通过配置参数能够自定义日期选择的细节，以下是对常用参数的使用说明：

配置参数	含义	备注
language	语言	zh-CN 代表中文
date	默认日期	
format	日期显示格式	y 代表年份，m 代表月份，d代表日份
startDate	起始日期	
endDate	截止日期	

- datepicker 默认语言为英文，通过 `language` 指定其它语言时需要引入对应的语言包（存放在 i18n 目录中），更多的配置参数详见官方文档。

- **插件——全屏滚动**
  - 全屏滚动是一种常见的网页特效，常被用于开发营销推广类网页，[fullPage](https://alvarotrigo.com/fullPage/zh/) 是一款十分优秀的用于开发全屏滚动网页的插件，其用法如下所示：
  - 首先引入插件及插件配套的样式：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 全屏滚动插件</title>
  <!-- 引入 fullPage 插件配套的样式 -->
  <link rel="stylesheet" href="./assets/fullpage/jquery.fullpage.min.css">
</head>
<body>

  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入 fullPage 插件 -->
  <script src="./assets/fullpage/jquery.fullpage.min.js"></script>
```

```
</body>
</html>
```

- 其次 fullPage 要求满足固定的 HTML 结构

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 全屏滚动插件</title>
  <!-- 引入 fullPage 插件配套的样式 -->
  <link rel="stylesheet" href="./assets/fullpage/jquery.fullpage.min.css">
</head>
<body>
  <div id="fullpage">
    <div class="section">第一屏</div>
    <div class="section">第二屏</div>
    <div class="section">第三屏</div>
    <div class="section">第四屏</div>
  </div>

  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入 fullPage 插件 -->
  <script src="./assets/fullpage/jquery.fullpage.min.js"></script>
</body>
</html>
```

- 上述中 HTML 结构中每个独立的【屏】默认要求其类名必须为 `section`。
- 再次调用 fullPage 插件方法：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>jQuery 全屏滚动插件</title>
  <!-- 引入 fullPage 插件配套的样式 -->
```

```
  <link rel="stylesheet" href="./assets/fullpage/jquery.fullpage.min.css">
</head>
<body>
  <div id="fullpage">
    <div class="section">第一屏</div>
    <div class="section">第二屏</div>
    <div class="section">第三屏</div>
    <div class="section">第四屏</div>
  </div>

  <script src="./assets/jquery/jquery-3.5.1.min.js"></script>
  <!-- 在 jQuery 之后引入 fullPage 插件 -->
  <script src="./assets/fullpage/jquery.fullpage.min.js"></script>
  <script>
    // 调用插件方法
    $('#fullpage').fullpage();
  </script>
</body>
</html>
```

- fullPage 也提供了若干的配置参数，现对常用配置参数做出说明：

配置参数	含义	备注
sectionsColor	为每【屏】设置的背景色	
navigation	是否显示指示器	
navigationPosition	指示器的位置	
afterLoad	切换【屏】后执行的回调	

- 更多的配置参数详见官方文档，fullPage v3.x 版本修改了开源协议，配合 jQuery 使用时建议采用 v2.x 的版本。

- **工具方法——遍历**

```
let arr = ['html', 'css', 'javascript'];
// 遍历数组，相当于 arr.forEach(function () {})
$.each(arr, function (val) {
  console.log(val);
```

```
})
```

- **工具方法——数组转换**

```
// 伪数组
let p = document.querySelectorAll('p');
// 报错，伪数组没有 pop 方法
// p.pop();

// 转换数组，相当于 Array.from(p)
let pArr = $.makeArray(p);
pArr.pop();
```

- **多库共存**
  - 实际开发中有可能存在同一个网页中引入不同版本 jQuery 的情形，不同版本之间彼此会产生覆盖现象，为了解决这个问题 jQuery 提供了 `$.noConflict` 方法。

```
<script src="./assets/jquery/jquery-3.5.1.min.js"></script>
<script>
  let _ = $.noConflict();
  // 重新命名 jQuery v3.5.1 为全局标识 _
  console.log(_.fn.jquery);
</script>
<script src="./jquery-2.2.4.js"></script>
<script>
  // jQuery v2.2.4 仍然使用全局标识 $
    console.log($.fn.jquery);
</script>
```

  - 利用 `$.noConflict` 方法还可以解决与其它类库（也以 $ 作为全局标识命名）全局标识命名冲突的问题。